



PDF Download
3719027.3765167.pdf
18 December 2025
Total Citations: 0
Total Downloads: 781

Latest updates: <https://dl.acm.org/doi/10.1145/3719027.3765167>

RESEARCH-ARTICLE

DiveFuzz: Enhancing CPU Fuzzing via Diverse Instruction Construction

ZIHUI GUO, Institute of Information Engineering, Beijing, China

MIAOMIAO YUAN, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China

YANQI YANG, Institute of Information Engineering, Beijing, China

LIWEI CHEN, Institute of Information Engineering, Beijing, China

GANG SHI, Institute of Information Engineering, Beijing, China

DAN MENG, Institute of Information Engineering, Beijing, China

Open Access Support provided by:

Institute of Information Engineering

Institute of Computing Technology Chinese Academy of Sciences

Published: 19 November 2025

[Citation in BibTeX format](#)

CCS '25: ACM SIGSAC Conference on
Computer and Communications Security
October 13 - 17, 2025
Taipei, Taiwan

Conference Sponsors:
SIGSAC

DiveFuzz: Enhancing CPU Fuzzing via Diverse Instruction Construction

Zihui Guo

Institute of Information Engineering,
Chinese Academy of Sciences
School of Cyber Security, University
of Chinese Academy of Sciences
Beijing, China

Miaomiao Yuan

Institute of Computing Technology,
Chinese Academy of Sciences
Beijing, China

Yanqi Yang

Institute of Information Engineering,
Chinese Academy of Sciences
School of Cyber Security, University
of Chinese Academy of Sciences
Beijing, China

Liwei Chen*

Institute of Information Engineering,
Chinese Academy of Sciences
School of Cyber Security, University
of Chinese Academy of Sciences
Beijing, China

Gang Shi

Institute of Information Engineering,
Chinese Academy of Sciences
School of Cyber Security, University
of Chinese Academy of Sciences
Beijing, China

Dan Meng

Institute of Information Engineering,
Chinese Academy of Sciences
School of Cyber Security, University
of Chinese Academy of Sciences
Beijing, China

Abstract

Comprehensive exploration of the CPU architectural states in fuzzing is akin to generating diverse test cases, which include a reasonable distribution of opcode and diversity in instruction execution results (typically measured through write-back data). However, our analysis of state-of-the-art CPU fuzzers reveals that they exhibit high repetition in write-back data and an imbalanced distribution of opcodes during fuzzing. This paper presents DiveFuzz, which diversifies write-back data by finely controlling the operands of instructions at runtime, coupled with correlated contextual semantics, to generate instruction streams with diverse write-back data and semantic associations. Furthermore, DiveFuzz introduces a novel mutator that monitors the fuzzing process to dynamically adjust opcode distribution and accurately eliminate false positives. Our evaluations show that DiveFuzz significantly increases the diversity of instruction write-back data and achieves a more balanced opcode distribution compared to state-of-the-art fuzzers. Across five common coverage metrics, DiveFuzz achieves coverage 204× faster than DifuzzRTL and 114× faster than Cascade. We evaluated DiveFuzz on four well-known open-source RISC-V CPUs—XiangShan, CVA6, Rocket, and NutShell—uncovering 26 new bugs, 15 of which have CVE identifiers.

CCS Concepts

• **Hardware** → **Hardware test**.

Keywords

Fuzzing, Hardware Verification, Test Generation

*Liwei Chen is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
CCS '25, Taipei

© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1525-9/2025/10
<https://doi.org/10.1145/3719027.3765167>

ACM Reference Format:

Zihui Guo, Miaomiao Yuan, Yanqi Yang, Liwei Chen, Gang Shi, and Dan Meng. 2025. DiveFuzz: Enhancing CPU Fuzzing via Diverse Instruction Construction. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25)*, October 13–17, 2025, Taipei. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3719027.3765167>

1 Introduction

The open-source and customizable characteristics of RISC-V have propelled it into the research spotlight across academia and industry. Yet, the ecosystem faces a pressing challenge: how to effectively verify the various RISC-V CPU designs to ensure their stability and security [6, 13].

In recent years, inspired by software fuzzing techniques [7, 24, 25, 32, 35, 51], CPU fuzzing techniques [9–12, 14, 22, 26, 27, 29, 31, 34, 45, 50, 52] have progressively emerged and gained attention. The common method for CPU fuzzing [9, 10, 12, 14, 22, 27, 29, 45, 50, 52] involves providing identical test inputs to both the CPU design and an Instruction Set Architecture (ISA) simulator (serving as a reference model) and then performing differential checking to verify if their architectural states (e.g., registers or memory) match. These state changes primarily arise from the instruction write-back process, where the results of executed instructions are written back to registers or memory to update the CPU state. Therefore, diversifying the write-back data means that more states can be examined, thereby verifying their correctness. Our study also demonstrates that increasing the diversity of write-back data not only triggers more bugs but also directly improves test coverage, as shown in Section 3.1. Furthermore, based on the principle of testing fairness, fuzzing should ensure a reasonable distribution of opcodes to avoid coverage imbalance of similar-function opcodes during testing. An ideal CPU fuzzer should focus on diversifying the instruction write-back data as much as possible while ensuring a balanced opcode distribution to achieve efficient testing.

However, the current CPU fuzzing ecosystem overlooks the two key issues mentioned above. Specifically, we analyze the state-of-the-art fuzzers [27, 45] and identified the following limitations: (i)

insufficient diversity in write-back data: the write-back data generated by these fuzzers exhibit high repetition, primarily due to the two-dimensional nature of instruction construction—namely, the selection of opcodes and operands. However, existing CPU fuzzers often lack a targeted approach when selecting operands, typically choosing them blindly without fully considering the actual impact of operands on instruction execution. Operand selection should take into account the update to the CPU state during instruction execution, but current tools fail to leverage this factor to increase the diversity of write-back data, leading to insufficient test coverage. (ii) imbalanced opcode distribution: this issue arises from factors such as instruction decoding optimizations, instruction fusion, and branching operations, which result in generated opcodes not matching the opcodes required for actual testing. This imbalanced distribution often leads to certain opcodes being overtested while others are neglected, introducing bias into the test results.

We introduce DiveFuzz, a novel CPU fuzzer designed to address the aforementioned challenges. DiveFuzz departs from traditional static generation methods that are solely based on instruction formats. Instead, it dynamically constructs instruction streams in real-time through interaction with an ISA simulator, optimizing operand distribution and integrating contextual information to generate diversified and semantically relevant instruction streams for fuzzing. Additionally, DiveFuzz monitors the actual execution frequency distribution of opcodes during fuzzing, autonomously adjusting the mutation trends to balance under-tested opcodes. Throughout the fuzzing process, false positives and known bugs often lead to test termination. Thanks to its real-time interaction system for obtaining instruction context, DiveFuzz can record opcode and operand key-value pairs associated with false positives and known bugs. These recorded key-value pairs are then precisely mutated in subsequent iterations, eliminating the impact of false positives and known bugs on testing and thereby improving the effectiveness of fuzzing.

We have developed a framework of DiveFuzz on the RISC-V architecture and evaluated it across a variety of RISC-V CPUs, spanning from simple to complex architectures and supporting various instruction extensions, including XiangShan [53], CVA6 [54], Rocket [5], and NutShell [48]. Our evaluation demonstrates that, compared to state-of-the-art fuzzers, DiveFuzz generates more diverse instruction streams with write-back data and achieves a more balanced opcode distribution. DiveFuzz outperforms state-of-the-art fuzzers across five coverage metrics—control reg, mux, line, toggle, and ready/valid—reaching the same coverage an average of 204× faster than DifuzzRTL [27] and 114× faster than Cascade [45]. During the fuzzing process, DiveFuzz identifies several known but unfixed bugs across four CPUs and uncovers new bugs in three of them.

In summary, our main contributions are as follows:

- We propose a dynamic runtime instruction operand construction method that efficiently enhances the diversity of instruction write-back data during fuzzing.
- We propose a systematic opcode balancing mutation strategy to ensure the comprehensiveness of fuzzing.
- We present the detailed design and implementation of DiveFuzz. DiveFuzz outperforms state-of-the-art fuzzers across

five coverage metrics, achieving the same coverage on average 204× faster than DifuzzRTL and 114× faster than Cascade.

- Our evaluation shows that DiveFuzz identified several known but unfixed bugs across four CPUs, uncovering a total of 26 new bugs, including 15 new CVEs.
- To support the community and foster future research, we make the source code of DiveFuzz available at <https://github.com/in2sec/DiveFuzz>.

2 Background

In this section, we present the features of the RISC-V instruction set and thoroughly examine the specific procedures and complexities involved in CPU fuzzing.

2.1 RISC-V

The RISC-V instruction set consists of a base set and various extensions. The base set includes operations such as integer arithmetic, control flow, memory access, and basic arithmetic functions, while the extensions provide capabilities like floating-point operations, vector processing, and cryptographic functions, which can be selectively implemented depending on application requirements. These optional extensions allow RISC-V to address the specific needs of diverse applications, reducing complexity and power consumption in embedded systems, while also supporting advanced features like vector processing and sophisticated memory management for high-performance computing [37, 54]. Notably, the RISC-V specification defines the behavior of these instructions but leaves their implementation open, resulting in different implementations across various RISC-V designs [1, 2, 19, 38]. RISC-V ISA simulators [43, 47] and hardware emulators [42] are indispensable tools for developing and testing RISC-V hardware systems, enabling software testing and debugging without the need for physical hardware. These tools model instruction execution, handle exceptions, and simulate hardware configurations, offering flexibility for hardware development and system validation.

The RISC-V architecture also defines multiple privilege levels: user mode for applications, supervisor mode for operating systems, hypervisor mode for virtualization, and machine mode for low-level hardware control. Control and status registers (CSRs) and privileged instructions manage these levels, storing critical state information and enabling interactions with these registers. This hierarchical structure ensures system security and stability by isolating operations at different privilege levels, thus preventing lower-privileged programs from interfering with higher-privileged ones.

2.2 CPU Fuzzing

Fuzzing techniques have demonstrated high effectiveness in the software testing domain [7, 24, 25, 35, 51]. Recently, hardware-based fuzzing has emerged as a valuable complement to traditional CPU verification methods. While conventional verification techniques such as functional simulation, formal verification, and test vector-based approaches are effective, they often face challenges such as high time consumption, limited coverage, and significant complexity [3, 17, 18, 23, 40]. In contrast, CPU fuzzing offers distinct advantages, particularly due to its ability to achieve high coverage

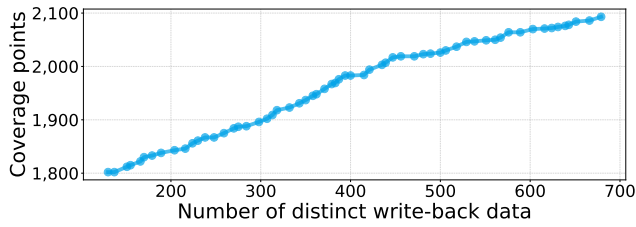


Figure 1: The relationship between the diversity of instruction write-back data and coverage.

Table 1: Some of the floating-point related bugs in CVA6.

CVE	Description	Error type
2023-34906	Incorrect handling of NaNs	RD write-back
2023-34905	Incorrect setting of NV bit	NV write-back
2023-34909	Incorrect setting of DZ bit	DZ write-back
2023-34907	Incorrect overflow handling	RD write-back
2023-34904	Incorrect rounding mode	RD write-back

and dynamically mutate test inputs, making it highly efficient for uncovering CPU bugs.

Existing CPU fuzzing techniques primarily rely on the concept of differential testing [9, 10, 12, 14, 27, 29, 45, 50]. In this approach, identical test inputs (seeds) are provided to both the CPU design under test (DUT) and the ISA simulator, which serves as a reference model (REF), and their outputs are compared for consistency. The output in this context refers to the instruction write-back data, which represents the final result produced by each executed instruction. Depending on the granularity of the write-back data checks, fuzzing can be categorized into program-level and instruction-level methods.

In program-level fuzzing, the comparison occurs after the entire assembly program has been executed. At this point, the final architectural state of the CPU—such as the registers and memory—is compared with the results from the ISA simulator. Since it only compares the final states, this approach may result in both false negatives and false positives. False negatives can occur when erroneous write-back data from intermediate instructions is overwritten by subsequent instructions. False positives, on the other hand, may arise from implementation differences between various RISC-V designs or from discrepancies introduced by the high configurability of RISC-V. Instruction-level fuzzing, in contrast, compares the write-back data of each instruction between the CPU and the ISA simulator immediately after the instruction is retired to determine if they match. This method offers finer granularity for error detection, effectively reducing false negatives, though false positives remain inevitable.

3 Motivation

In this section, we first present case studies to highlight the importance of instruction write-back data diversity in fuzzing. We then analyze the phenomena observed in state-of-the-art fuzzers DifuzzRTL [27] and Cascade [45]. Based on these observations, we

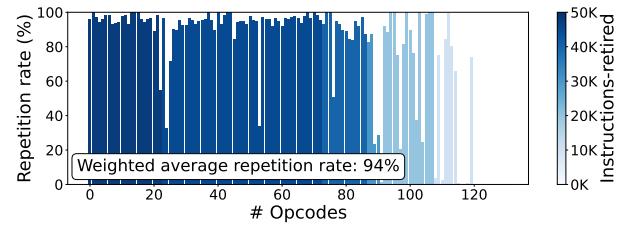


Figure 2: Instruction write-back data repetition rate of Cascade. To provide a clearer view, we sorted the opcode sequence based on the number of instructions.

identify and articulate the key challenges involved in the design of DiveFuzz.

3.1 Case Study

To illustrate the impact of the diversity of instruction write-back data on the DUT coverage and bug detection during CPU fuzzing, we conduct a case study to examine their relationship.

As shown in Figure 1, the number of distinct instruction write-back data (diversity) affects the DUT coverage. It is important to note that, to isolate the impact of this single factor, we diversified the instruction write-back data solely by modifying the operands, without changing the number of instructions or opcodes contained in the test cases. Therefore, the leftmost data point in Figure 1 represents the state of the test cases before any modifications. As shown in the figure, the number of distinct instruction write-back data appears to have a positive correlation with DUT coverage.

Next, we use bugs in the CVA6 CPU as case studies to demonstrate the relationship between write-back data and bug triggers. As shown in Table 1, the table lists several floating-point instruction-related bugs, with the last column displaying the error information of the write-back data, specifically the register bits where the error occurs. It is important to note that all the bugs in these cases can be triggered by identical floating-point instructions (e.g., `fdiv`) with different write-back data. Therefore, during fuzzing, diversifying the write-back data of instructions can increase the variety of test scenarios, potentially leading to the discovery of more bugs.

Conclusion. Diversifying the instruction write-back data can increase the coverage of the DUT and potentially uncover more bugs.

3.2 Observations

We use an ISA simulator [43] to capture the diversity of write-back data in the test cases generated by the fuzzers. Figure 2 presents the repetition rate of the write-back data for approximately two million instructions generated by Cascade. The x-axis represents different opcode operations, while the y-axis indicates the repetition rate of the write-back data. The darker the color of the bars, the higher the frequency of these instructions. The annotations represent an average repetition rate of write-back values of approximately 94%. From the overall analysis, it is evident that the write-back data repetition rate for most instructions is approximately 90%, even though these instructions theoretically have a large exploration

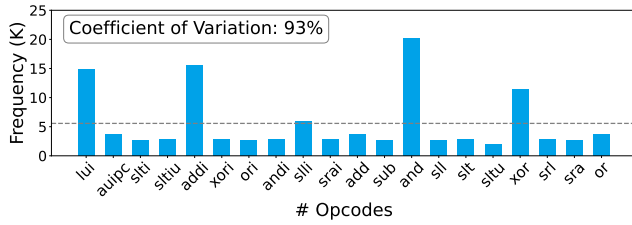


Figure 3: Instruction frequency distribution of the RV32I extension in Cascade during the real fuzzing of 2 million instructions, with the dashed line representing the average frequency.

space. Under similar conditions, DifuzzRTL shows an average repetition rate of 93%. Because it is difficult to predict the write-back behavior during instruction construction, current fuzzers focus solely on syntax when generating test instructions, which leads to the aforementioned high repetition rate of write-back data.

O1. CPU fuzzers encounter obstacles in generating test instructions with diverse write-back data.

To maintain the principle of test fairness, current fuzzers ensure that the opcodes within an instruction extension/category are uniformly tested by assigning a random selection factor to the instruction's extension/category. In the two million instructions generated by Cascade, we analyze the actual fuzzing frequency of arithmetic instructions in the RV32I extension. As shown in Figure 3, there is a significant difference in the fuzzing frequencies of these instructions. The coefficient of variation (CV), a common measure of relative variability, is the ratio of the standard deviation to the mean, and is approximately 93%. Under the same conditions, the coefficient of variation for DifuzzRTL is 96%. This exceptionally high CV indicates an imbalance in fuzzing, resulting in some opcodes remaining in a relatively "starved" state for extended periods. Our analysis reveals that, although theoretically uniform opcode distribution can be achieved by assigning random weights, factors such as branch instructions, instruction decoding optimizations, pseudo-instructions, and early test termination lead to a significant discrepancy between the generated instructions and the instructions actually fuzzed.

O2. Current CPU fuzzing suffers from imbalanced opcode distribution.

Summary. Highly repetitive instruction write-back data and imbalanced opcode distribution significantly limit the effectiveness of current fuzzers.

3.3 Challenges

C1: How to diversify the write-back data of instructions?

The values of instruction operands (e.g., registers, memory data) directly affect the write-back data. However, existing static instruction generation techniques fail to dynamically capture these values,

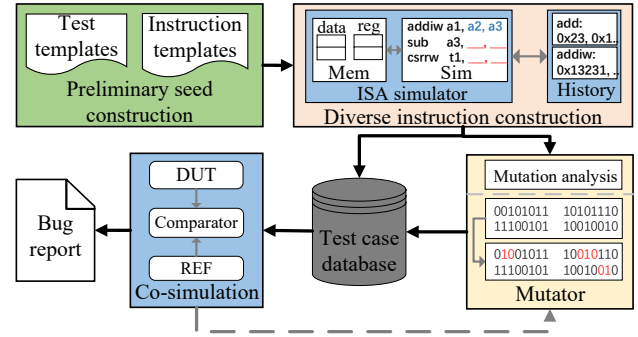


Figure 4: Overview of DiveFuzz.

making it impossible to create diversified instruction operands. To address this challenge, we interact with the ISA simulator in real time to retrieve runtime information during the instruction stream construction process. This allows for the dynamic adjustment of operand distribution, enabling diversification of the write-back data. However, focusing solely on the diversification of write-back data in the instruction stream may result in a lack of semantic richness and limit contextual relevance, ultimately reducing the potential for triggering complex bugs [15, 16]. Our goal is to ensure that the diversified instruction stream incorporates rich semantic scenarios. Section 4.3 details an instruction generation strategy designed for CPU fuzzing to enhance the diversity of write-back data.

C2: How to balance instruction opcodes?

Section 4.4 introduces a fuzzing mutation strategy aimed at balancing opcode distribution. This approach monitors the actual execution frequency distribution of opcodes and performs mutations on existing test cases. The straightforward method of mapping frequency to probability is highly susceptible to extreme values [8], which may lead to an imbalanced opcode distribution in subsequent fuzzing rounds. Our design goal is to ensure that the opcodes under test are similar to the set expected values, guaranteeing balanced testing for each opcode within the extension, thereby enhancing the reliability of the fuzzing process.

4 DiveFuzz Design

In this section, we elaborate on the features and design details of DiveFuzz. We start with an overview and then delve into the detailed design of each component. Figure 4 shows an overview of DiveFuzz.

4.1 Approach Overview

The DiveFuzz workflow consists of five steps: 1) preliminary seed construction, 2) dynamic diverse instruction construction, 3) Co-simulation testing between the CPU and REF, 4) seed mutation (balancing the opcode distribution after fuzzing), and 5) bug reporting.

Preliminary seed construction. During the preliminary stage, DiveFuzz constructs test templates that includes a testing environment, equipped with instruction templates covering various

instruction extensions. In this phase, the primary task is the construction of static seeds. Specifically, the test templates provide infrastructure support such as configurations for different privilege modes, data heap population, and page table settings. The instruction templates define the field layouts for various opcodes. It is worth noting that, unlike prior fuzzers, this stage generates only the opcode without populating its operands; that is, the output is an incomplete instruction. This deferred operand construction is designed to enable targeted diversification of write-back data, which requires constructing the full instruction based on runtime context.

Diverse instruction construction. DiveFuzz constructs diversified input seeds, focusing on two key aspects: diversified instruction write-back data and the generation of instruction streams with contextual relevance. After obtaining the preliminary seeds, DiveFuzz gradually fills in the incomplete seeds for dynamic interaction with the ISA simulator to capture the context information (e.g., memory and register values) of the instructions, thereby generating instruction streams with diversified write-back data in real-time. Additionally, it leverages contextual information to enhance the semantic relevance of generated instruction streams. The entire process adopts a parallelized interaction architecture to reduce time overhead.

Co-simulation testing. DiveFuzz implements a co-simulation framework that can dynamically compare the write-back data at the instruction level between the DUT and REF in real-time. When the data matches, the simulation continues; however, when inconsistencies are detected, they are considered potential bugs, requiring subsequent manual analysis to further identify the root cause of the issue.

Seed mutation. Seed mutation primarily involves analyzing the execution history from the previous fuzzing round to adjust the frequency of opcode occurrences in subsequent rounds. DiveFuzz utilizes mathematical constraints to calculate the mutation probabilities and tendencies for each opcode, thus achieving a balanced distribution of opcodes. Additionally, by leveraging real-time execution information, the mutator is responsible for handling test interruptions caused by certain false positives and known bugs in subsequent fuzzing rounds, ensuring deeper fuzzing and increasing the testing of the generated instruction.

4.2 Preliminary Seed Construction

DiveFuzz offers a rich set of optional testing environment configurations and instruction extensions (refer to Appendix A). During the preliminary seed construction stage, DiveFuzz first selects a test template from the available options to provide environmental configurations for CPU fuzzing. These configurations offer an environment for CPU fuzzing that operates independently from the operating system, including settings for exception handling, page table configurations, and privilege mode settings. Based on the selected test template, DiveFuzz progressively fills in the tested opcodes according to preset configurations, leaving the operands empty. Subsequently, random bytecode is used to pseudo-fill the operands, forming a complete instruction. It is important to emphasize that these pseudo-filled instructions are not the final test

Listing 1: Example of fuzzing code.

1	<code>div</code>	<code>t0, a1, a2</code>	<code># instr1</code>
2	<code>div</code>	<code>t1, a2, a1</code>	<code># instr2</code>
3	<code>csrrw</code>	<code>t3, instret, t2</code>	<code># instr3</code>
4	<code>csrrw</code>	<code>t3, mtval, t2</code>	<code># instr4</code>

instructions; they merely serve as placeholders and do not affect the testing results in the early stages. In subsequent stages, DiveFuzz interacts with the ISA simulator to complete the operands of these instructions with the goal of diversifying the write-back data.

4.3 Diverse Instruction Construction

We elaborate on the efficient construction of instruction streams with contextual semantics and diversified write-back data.

4.3.1 Diverse instructions. The entire input construction process dynamically builds instruction streams with diversified write-back data and correlated contextual semantics to improve both the breadth of testing and the depth of instruction stream semantics. First, DiveFuzz diversifies the instruction write-back data by varying the distribution of operands. Specifically, it places the preliminary seeds obtained in the previous step into the instruction memory of the ISA simulator, simulating the real-time context of the tested instructions, including all register and memory values. After the environment set up by the test template completes execution, the interactive module uses the real-time instruction context to progressively modify the operands of the pseudo-filled instructions from the previous stage, ensuring as much diversity as possible in the constructed instruction operands. However, since different instructions require different numbers of operands, directly recording and comparing all operand values for multi-operand instructions is inefficient and may overlook the sensitivity of operand order. Although XOR-based duplicate checking appears promising, it fails to meet key test coverage requirements. For example, as shown in Listing 1, the operands `a1` and `a2` in instructions `instr1` and `instr2` generate the same XOR value but semantically correspond to entirely different division instructions. To address this, DiveFuzz introduces the Shift-XOR technique (see Equation 1), which shifts operand values based on their positions and order before performing the XOR operation. The resulting XOR value, V , serves as the unique identifier for duplicate tracking while maintaining sensitivity to operand order. It is noteworthy that this technique imposes no restriction on the bit width of the shift value, ensuring data integrity while accommodating various operand combinations. Although this increases memory requirements to some extent, it provides flexibility in representing diversified operand configurations. Furthermore, DiveFuzz maintains a global history of V values for different instruction types, which is rigorously checked for diversity in subsequent mutation phases. For privileged instructions, DiveFuzz classifies the opcodes in conjunction with CSR names. For example, `instr3` and `instr4` share the same `csrrw` opcode but operate on different CSRs, and are thus considered distinct instruction types.

It is worth noting that, after implementing the aforementioned strategy for diversifying write-back data, DiveFuzz successfully

identified all the floating-point related bugs listed in Table 1 within a few hours, as well as additional floating-point related bugs B10 to B13. These bugs, namely B10 and B13, were theoretically discoverable by previous fuzzers [27, 45]; however, due to the lack of diversity in write-back data, the relevant instruction execution scenarios were not effectively explored during the two-week period of using the prior fuzzer.

$$V = (\text{operand}_N \ll N) \oplus \dots \oplus (\text{operand}_1 \ll 1) \oplus \text{operand}_0 \quad (1)$$

4.3.2 Enhance semantic relevance. To enhance the semantic relevance of the generated instruction streams, DiveFuzz introduces two global priority queues during the selection of registers for operands: the recently used destination registers (RURD) and the recently used source registers (RURS). These queues dynamically maintain the usage history of registers during the instruction generation process and are updated using the least recently used (LRU) replacement algorithm. To improve instruction construction efficiency, DiveFuzz adopts the following strategies: when filling source registers, it increases the likelihood of selecting from the RURD set to ensure continuity in the data flow; when selecting destination registers, it favors registers from the RURS set to enhance contextual relevance. This cross-selection strategy enhances the semantic relevance within the generated instruction stream and demonstrates improved potential in triggering various types of CPU hazards during testing. As shown in Listing 2, the example shows a code snippet generated using the strategy, along with the corresponding updates to the RURD and RURS queues during instruction construction and the types of CPU hazards that were triggered. The blue annotations in the listing indicate the specific types of CPU hazards encountered. Building on the diversification of write-back data, this strategy enabled the successful discovery of bug B13.

During the construction process, once an instruction's format is determined (i.e., the pseudo-operands are finalized), the RURD and RURS queues are updated in real-time based on the latest instruction information. To support more comprehensive testing, DiveFuzz periodically refreshes the two queues after generating a certain number of instructions, helping to mitigate potential bias in register selection. Additionally, DiveFuzz introduces a boundary testing mechanism during operand construction, probabilistically generating boundary cases (e.g., extreme immediate values or special register values). This design aims to trigger boundary conditions within the CPU architecture, potentially exposing latent defects. When the RURD and RURS queues are unable to provide sufficiently diverse operand values, DiveFuzz expands the selection range by choosing registers outside the queues to ensure operand diversity. Although this strategy may slightly reduce the semantic relevance of the instruction stream, it significantly increases the diversity of write-back data in the test inputs. It should be noted that while this dynamic instruction construction approach introduces some time overhead compared to previous static techniques, the impact on the overall fuzzing process is minimal, as the ISA simulator operates several orders of magnitude faster than the DUT.

Listing 2: Instruction sequence with hazard annotations.

```

1  ...
2  add      x7, x5, x6          # instr5
3  # -> RURD = [x7], RURS = [x5, x6]
4
5  sub      x8, x7, x5          # instr6
6  # -> RURD = [x7, x8], RURS = [x5, x6, x7]
7  # -> Hazard: RAW on x7 (data dependency on instr5)
8
9  slt      x7, x6, x8          # instr7
10 # -> RURD = [x7, x8], RURS = [x5, x6, x7, x8]
11 # -> Hazard: RAW on x8 (data dependency on instr6)
12 # -> Hazard: WAR on x7 (overwrites instr6 operand)
13
14 xor      x7, x5, x1          # instr8
15 # -> RURD = [x7, x8], RURS = [x5, x6, x7, x8, x1]
16 # -> Hazard: WAW on x7 (overwrite of instr7 result)
17  ...

```

4.4 Co-simulation Testing

To maximize the verification of instruction write-back data, DiveFuzz adopts a fine-grained instruction-level runtime verification mechanism, performing verification of the matching write-back data between the DUT and the REF on a per-instruction basis. The actual executed opcode stream is fed back to the fuzzer in real-time, allowing adjustments to the distribution of subsequent opcodes. During fuzzing, when the write-back data of the DUT and REF are confirmed to match, the execution flow seamlessly continues; however, when mismatches are detected, they are considered potential bugs and subjected to manual analysis.

However, not all mismatches indicate the presence of a bug, as false positives are inevitable during fuzzing. These false positives not only hinder the testing process but may also waste generated seeds. To mitigate such interference, DiveFuzz records all false-positive instructions along with their corresponding opcodes and operands during the subsequent bug analysis phase. Thanks to DiveFuzz's runtime instruction construction, when encountering the same opcode in future testing, we can avoid constructing operands that would lead to false positives. Unlike previous approaches [27, 45] that entirely avoid generating a specific opcode, DiveFuzz does not require discarding all possible test cases but instead precisely avoids triggering known bugs, thus avoiding false positives and repeated triggering of known defects without sacrificing testing breadth.

4.5 Seed Mutation

The mutator performs two core functions. First, it adjusts the mutation of opcodes based on the deviation between the opcode frequency feedback from the co-simulation framework and the preset expected frequency, thereby balancing the distribution of subsequent opcodes with similar functionality. Second, it performs fine-grained mutations at the operand level to effectively eliminate known bugs or recorded false-positive information, ensuring the comprehensiveness and accuracy of the testing.

4.5.1 Opcode balance. DiveFuzz analyzes the seeds from the previous fuzzing round, quantifies the execution frequency of each instruction type, and maps it to a probability model to dynamically adjust the frequencies. The fuzzer first performs cluster analysis on the opcode frequencies within the instruction set extension, categorizing them into frequent instructions (FI) and rare instructions (RI) based on their average occurrence. During the mutation process, RI are retained, and their operand diversity is optimized to ensure coverage of rare opcodes. For FI, some are probabilistically replaced with RI, while diversifying the operand distribution. By adjusting opcode frequency during fuzzing, this approach prevents over- or under-testing of certain instruction types. To quantify frequency deviations and smooth the mutation probability distribution, DiveFuzz uses Z-score normalization, standardizing the opcode frequencies as a measure of deviation from the mean. The formula is as follows:

$$z = \frac{f_i - \mu}{\sigma}.$$

In this formula:

- f_i represents the number of occurrences of the i -th opcode.
- μ (the mean) represents the average number of opcode occurrences within each extension in the dataset, calculated as:

$$\mu = \frac{\sum_{i=1}^n X_i}{n},$$

where X_i represents the number of occurrences of a type opcode, and n represents the number of opcode types within the extension.

- σ (the standard deviation) measures the amount of variation or dispersion of opcode occurrences from the mean, calculated as:

$$\sigma = \sqrt{\frac{\sum_{i=1}^n (X_i - \mu)^2}{n}}.$$

However, since Z-scores lack probabilistic properties, DiveFuzz applies a logistic transformation to map Z-scores to the (0,1) interval. The logistic function not only bounds probabilities but also optimizes probability management robustness by adjusting scale and distribution shape. Specifically, the probability of increasing or decreasing instruction frequencies is calculated as follows:

$$P_{\text{increase}} = \frac{1}{1 + e^z},$$

$$P_{\text{decrease}} = \frac{1}{1 + e^{-z}}.$$

This approach ensures the consistency of the probability direction and the smooth adjustment of its magnitude. Through these transformations, DiveFuzz establishes a dynamic and fine-grained instruction frequency adjustment mechanism to enhance the reliability of fuzzing.

4.5.2 Circumventing false positives and known bugs. False positives are common in CPU fuzzing. For example, the RISC-V specification allows the `wfi` instruction to be implemented as a `nop` instruction or as a `wait-for-interrupt` instruction. Different implementations can lead to varying execution behaviors. Additionally, different CPUs have different implementations for CSR selection, meaning that during CSR access, the REF may succeed while the CPU fails,

leading to differences in write-back data and causing the fuzzing process to terminate. Furthermore, as bug fixes in the CPU can be time-consuming, it is crucial to avoid terminating the testing process due to known bugs. To address these issues, DiveFuzz employs a feature-based knowledge base composed of opcode-operand key-value pairs. All false positives or known bugs are recorded as these key-value pairs. During the real-time mutation process, the ISA simulator provides the instruction context, enabling verification of whether the instruction matches any key-value pairs in the knowledge base. If a match is found, the operands of the instruction are mutated accordingly. It is important to note that although the knowledge base is constructed gradually, it requires minimal manual intervention—only opcodes and corresponding operands need to be populated. For example, by recording floating-point division by zero, DiveFuzz can avoid triggering division-by-zero bugs in subsequent testing, ensuring that bugs are not missed due to prematurely interrupting the test.

5 Evaluation

In this section, we evaluate DiveFuzz across multiple aspects, focusing on the challenges **C1** and **C2** defined in Section 3.3. We present experimental results on instruction write-back data repetition rates, diversity, and fuzzing balance, alongside evaluations of coverage metrics, bug detection capabilities, and component analysis.

Sections 5.2 and 5.3 address **C1** and **C2**, respectively, by analyzing instruction write-back data diversity, repetition trends, and mutation-driven frequency balance. Section 5.4 compares coverage across five metrics—control reg, mux, line, ready/valid, and toggle—while Section 5.5 highlights DiveFuzz’s bug detection capabilities. A security analysis of representative bugs is presented in Section 5.6. In Section 5.7, we evaluate its efficiency in false positive handling. Section 5.8 presents an ablation study of its components.

5.1 Experimental Setup

Our experiments were conducted on a machine equipped with an AMD EPYC 9684X processor operating at 3.7 GHz, featuring 384 logical cores and 1 TB of DRAM. We utilized Verilator [42] for hardware emulation, version v5.018. The ISA simulator used is Spike [43], version 1.1.1-dev, commit f8b2e39. During fuzzing on XiangShan, we used the vendor-provided customized ISA simulator, Nemu (commit 277ac7d), as the differential comparison tool. The operating system environment is Ubuntu 22.04. We fuzzed various CPUs including XiangShan [53], CVA6 [54], Rocket Core [5], and NutShell [48], with their specific attributes detailed in Appendix B. In evaluating coverage, all instrumentation tools are sourced from [27, 30, 31]. Regarding the co-simulation platform for testing, we modified the test platforms provided by these repositories [36, 39] to accommodate the design objectives of DiveFuzz. Due to open-source limitations, we selected DifuzzRTL [27] and Cascade [45] as comparison baselines. Nevertheless, this selection is well-justified, as most fuzzers use DifuzzRTL as a benchmark (see Table 5), and Cascade represents one of the most recent advancements in this field. All evaluation results were based on single-core performance; however, during the actual fuzzing phase, we leveraged multi-core parallelism to improve testing efficiency. We implemented approximately 8K lines of Python code to build DiveFuzz.

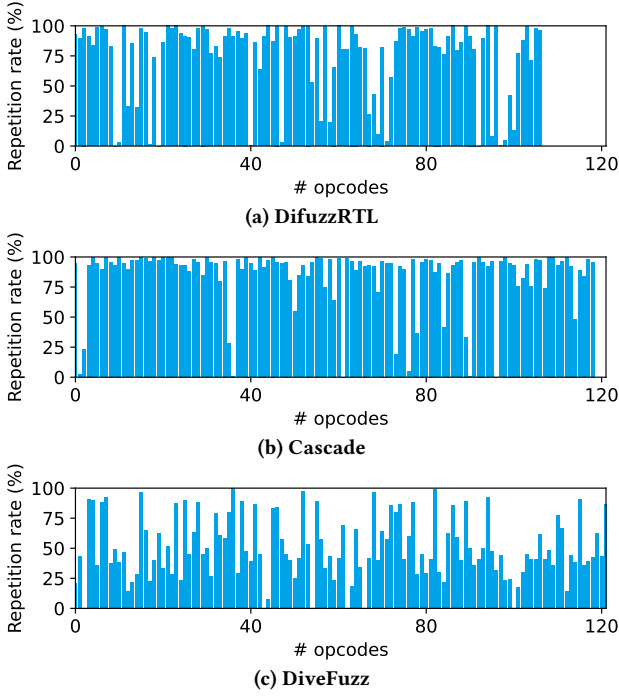


Figure 5: Repetition rate of write-back data corresponding to instructions for DifuzzRTL, Cascade, and DiveFuzz when fuzzing 2 million instructions.

5.2 Instruction Write-back Data Diversity

We evaluated the effectiveness of the solution **C1** proposed in Section 3.3 through three experiments, further demonstrating the advantages of DiveFuzz in terms of instruction diversity for fuzzing. First, we compared the write-back data repetition rates for each instruction type among DifuzzRTL, Cascade, and DiveFuzz, as shown in Figure 5. The x-axis represents different opcodes, while the y-axis indicates the write-back data repetition rate for each opcode. As can be seen from the figure, DiveFuzz effectively reduces the repetition rate of write-back data by diversifying the distribution of instruction operands.

Furthermore, we calculated the weighted average of these repetition rates, which also serves as the core data source for **O2** in Section 3.2. The use of weighted averages aims to provide a more accurate evaluation, particularly by accounting for the significant differences in the number of tests conducted on the same instruction types by DifuzzRTL and Cascade, thereby mitigating their impact on the accuracy of repetition rate calculations. As shown in Figure 6, we compared the weighted repetition rates of the three fuzzers. The results indicate that during the early stages of fuzzing, the repetition rates of DifuzzRTL and Cascade already reached approximately 90% and continued to rise as testing progressed. In contrast, DiveFuzz exhibited fluctuating repetition rates throughout the fuzzing process, likely due to imbalanced opcode distributions in the initial stages. Notably, DiveFuzz maintained a superior repetition rate of approximately 55%. It is important to note that certain instructions inherently produce write-back data with only single

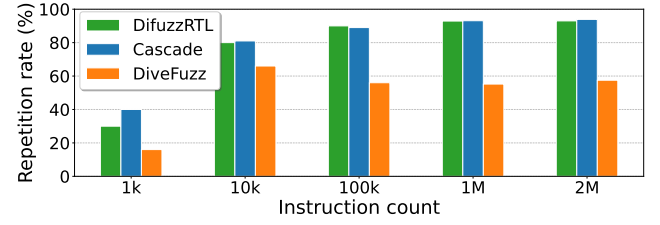


Figure 6: Comparison of average repetition rates of write-back data for DifuzzRTL, Cascade, and DiveFuzz when fuzzing different numbers of instructions.

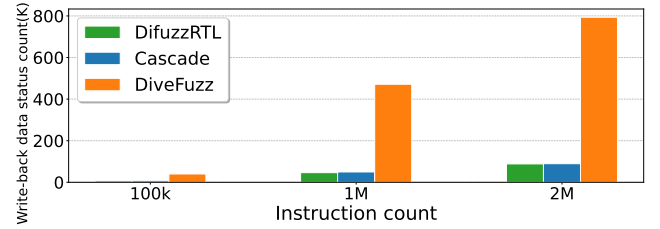


Figure 7: Comparison of the diversity of write-back data for DifuzzRTL, Cascade, and DiveFuzz when fuzzing different numbers of instructions.

values, such as always writing back 0 or 1. This imposes a natural limitation on the ability to reduce redundancy, preventing very low repetition rates from being achieved.

Finally, to illustrate the relationship between repetition rates and diversity, we evaluated the write-back data diversity of the three tools during fuzzing. As shown in Figure 7, the y-axis represents the total number of unique write-back data states. The results demonstrate that DiveFuzz exhibits a significant advantage in write-back data diversity compared to DifuzzRTL and Cascade, enabling the verification of a broader range of CPU states during fuzzing.

5.3 Opcode Balance

We evaluated the balance of opcode distribution during the fuzzing process by calculating the coefficient of variation (CV) of the opcode distribution. A smaller CV value indicates a more balanced distribution of opcodes. To ensure consistency with Section 3.2, we selected basic arithmetic instructions supported by all three fuzzers for comparison. These instructions have similar functionalities and, under ideal conditions, should be tested evenly. We evaluated the CV of DiveFuzz both before and after mutation, with the results shown in Figure 8. As described in Section 3.2, both DifuzzRTL and Cascade exhibit high CV values, indicating significant imbalance in instruction distribution during fuzzing. In contrast, supported by its mutator, DiveFuzz achieves a more desirable balance in instruction distribution. Interestingly, before mutation, DiveFuzz's CV results are similar to those of DifuzzRTL and Cascade. However, after mutation, DiveFuzz successfully balances fuzzing by converting over-tested opcodes into under-tested ones, reducing the CV from 93% to 30.3%. Our analysis shows that all three fuzzers exhibit similar CV results before mutation, primarily due to the presence

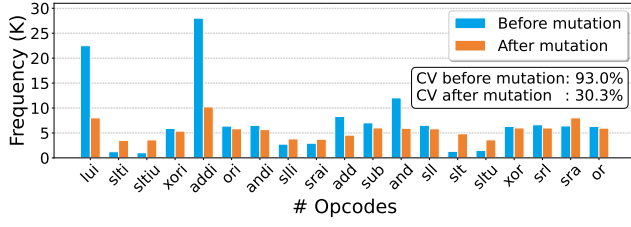


Figure 8: Comparison of the actual fuzzing execution frequencies of RV32I arithmetic instructions before and after mutation by DiveFuzz.

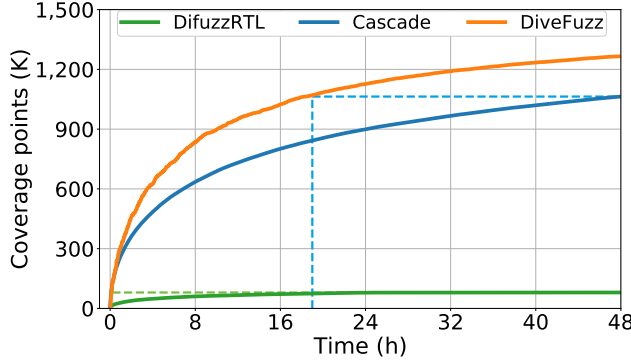


Figure 9: Comparison of the control-register coverage for DifuzzRTL, Cascade, and DiveFuzz over time.

of fused instructions, decode optimizations, branch instructions, and pseudo-instructions. These instructions are decomposed into multiple instructions during the decode stage, and some implementations may optimize this process during decoding. This explains why, despite assigning relatively uniform probability factors during instruction generation, significant differences in actual execution probability distributions remain.

5.4 Coverage Evaluation

To evaluate DiveFuzz’s performance in terms of coverage, we compared DifuzzRTL, Cascade, and DiveFuzz using five commonly used coverage metrics in Rocket: control-registers, multiplexers (mux), lines of code (line), ready/valid signals (ready/valid), and signal toggles (toggle). As shown in Figures 9 and 10, DiveFuzz outperforms the other fuzzers across all five coverage metrics.

To ensure cross-platform compatibility, we statically generated an equal number of seeds for all three fuzzers and collected coverage data sequentially. The minor differences in static seed generation speeds among the fuzzers are negligible compared to the evaluation duration, making the resulting deviation acceptable. Among the five coverage metrics, four (mux, line, ready/valid, and toggle) reached saturation relatively quickly. In contrast, the control-register metric required extended data collection due to the large number of potential coverage points. Specifically, we collected coverage data for up to 48 hours for control-registers, while for the other four metrics, which have fewer coverage points and saturate more quickly,

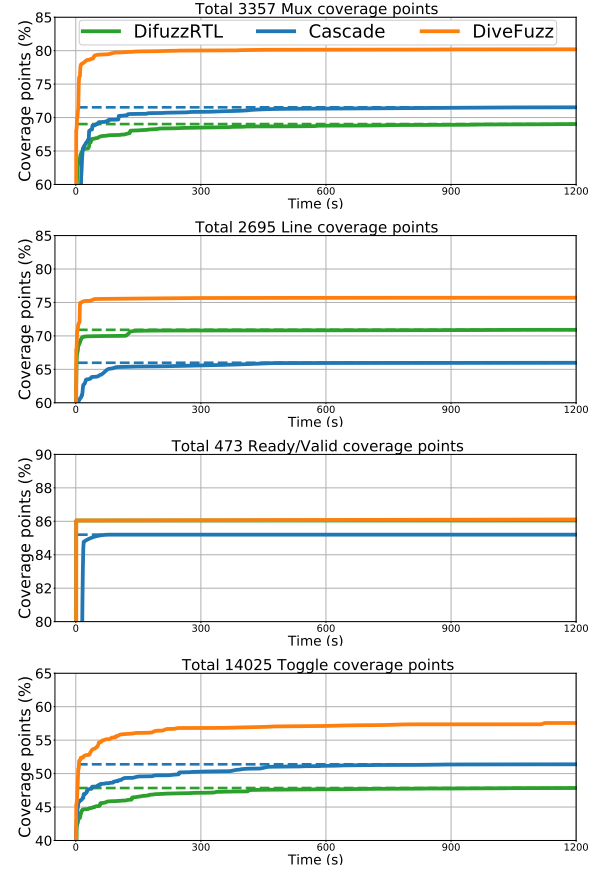


Figure 10: Comparison of the coverage of mux, line, ready/valid, and toggle for DifuzzRTL, Cascade, and DiveFuzz over time.

we limited the collection period to 1200 seconds. As shown in Table 2, the first row lists the different coverage metrics, while the remaining rows show the speedup of DiveFuzz in achieving coverage compared to the other two fuzzers. In summary, by utilizing pre-seeds, DiveFuzz achieves the same coverage 204× faster than DifuzzRTL and 114× faster than Cascade. Furthermore, DiveFuzz attains the highest coverage across all five metrics. During the evaluation process, for collecting control register coverage, we used a specific version of Rocket supported by DifuzzRTL for control register coverage. For the other four coverage evaluations, we utilized the latest Rocket commit version.

5.5 Bugs Discovered

We conducted a statistical analysis of all previously unknown bugs identified by DiveFuzz, with the results summarized in Table 3. The table records the affected designs (including three CPUs and two ISA simulators), a brief description of each bug, and their current status (fixed, confirmed, or submitted to the vendor). It also includes the CVE or issue identifier and the corresponding CWE number for each bug. As shown in the table, DiveFuzz successfully identified 26

Table 2: Comparison of coverage speed between DiveFuzz and the other two fuzzers. Con refers to control registers, R/V to ready/valid, and Tog to toggle coverage metrics.

Fuzzer	Con	Mux	Line	R/V	Tog	Average
DifuzzRTL	288x	328x	205x	1x	199x	204x
Cascade	2.5x	154x	255x	60x	103x	114x

previously unknown bugs. We then classify these bugs and provide detailed descriptions for each. To ensure a fair comparison across fuzzers on the same DUT and to better demonstrate the effectiveness of DiveFuzz’s instruction-level diversity in uncovering bugs, we conducted an additional bug-finding evaluation, as presented in Appendix C. All identified bugs were reported to the respective development teams, and we collaborated proactively to resolve them.

5.5.1 Calculation Bugs. These bugs involve logical flaws in the CPU’s processing of instructions or data, leading to inaccurate results that may cause program behavior anomalies or computational errors.

B1. The `flh` instruction fails to perform proper sign extension, resulting in incorrect handling of negative floating-point values and violating the IEEE 754 standard.

B6. Errors in rounding mode propagation lead to mismatches in fused multiply-add (FMA) results, potentially affecting precision-critical tasks.

B10. Certain microarchitectures exhibit rounding mode calculation errors, deviating from standard rounding expectations.

B11. The overflow flag (OF bit) is incorrectly set during `fsqrt` (floating-point square root) operations, potentially causing software to misjudge floating-point exceptions.

B12. Errors in handling the smallest negative number and negative infinity could result in failures in edge case management during mathematical computations.

B13. Certain instruction sequences fail to correctly set the NX (inexact) flag, impacting the evaluation of computational accuracy.

B14. The `next.h` instruction is incorrectly implemented, causing data width processing to deviate from expectations.

B18–19. Atomic instructions incorrectly handle the sign bit, potentially leading to inconsistencies in concurrent operations and affecting thread synchronization.

5.5.2 Access Control Bugs. These bugs involve flaws in access control mechanisms, potentially allowing unauthorized access or bypassing permission checks, posing security risks.

B2. Missing privilege checks for the `stopi` CSR in the Advanced Interrupt Architecture (AIA) may allow low-privilege code to illegitimately control interrupt behavior.

B3. Access control errors in custom CSRs under U-mode enable low-privilege users to modify high-privilege states.

B7. High-bit address truncation in the SV39 extension bypasses Physical Memory Protection (PMP) checks, enabling unauthorized memory access.

B17. Logical errors in address truncation during memory access disable boundary checks, potentially causing memory leaks or out-of-bounds access.

B22. Incorrect implementation of the LCOFI bit in `hvip` allows unauthorized state changes.

B23. Misconfiguration of the GVA (Guest Virtual Address) bit in `mstatus` may lead to inconsistencies in virtualized environments.

B24. Dependency errors in the `henvcfg` CSR affect correct access control for specific configurations.

B25. The LCOFI bit in `hvip` incorrectly depends on irrelevant signals, causing privilege management issues.

B26. Dependency errors in the `mstateen0` CSR may allow unauthorized state writes.

5.5.3 Exception Handling Bugs. These bugs involve flaws in the implementation of exception handling mechanisms, potentially leading to failed exception signals or incorrect recovery.

B4. Redundant checks when accessing S-mode custom CSRs in VS-mode cause performance degradation and increase processing complexity.

B20. Page faults in M-mode fail to update registers correctly, propagating erroneous states and impacting subsequent operations.

B21. Logical errors in M-mode exception handling prevent proper hardware exception capture or recovery.

5.5.4 Interaction Logic Bugs. These bugs arise from flaws in the interaction logic between different hardware or software components, leading to inconsistent states or functional abnormalities.

B5. Interaction logic errors between `xstatus`, `SDT` and `sie` result in inconsistencies during state register transitions.

B22. Incomplete interaction logic between the LCOFI bit in `hvip` and other signals affects correct interrupt handling.

B23. Dependency errors in the GVA bit of `mstatus` cause address handling inconsistencies in virtualized environments.

B26. Interaction logic errors in the `mstateen0` register’s control signals may result in improper configurations.

5.5.5 Configuration and State Management Bugs. These bugs involve improper handling of configuration registers or state management, potentially leading to functional abnormalities or the propagation of erroneous states.

B8. Errors in the propagation logic of special floating-point operands (e.g., NaN) cause register states to deviate from expectations.

B9. Memory access beyond boundaries is permitted, leading to potential memory leaks or data corruption.

B15. Incorrect configuration of `mtvec/stvec` CSRs causes errors in exception vector handling.

B16. Incorrect handling of the WPRI (Reserved Writes Preserve Values, Reads Ignore Values) bits in the `mstatus` register may propagate erroneous states.

5.5.6 Other Bugs. DiveFuzz also identified several known but unfixed bugs, including a bug in Rocket where the `ebreak` and `ecall` instructions incorrectly increment the `minstret` register, and several floating-point computation bugs in CVA6. Our findings indicate that DiveFuzz not only discovers new bugs but also successfully identifies many previously reported but unfixed bugs found by other fuzzers. Since these bugs have been documented in prior work, we do not elaborate on them here.

Table 3: A list of new bugs discovered by DiveFuzz, noting that known bugs are not included.

Design	Bug Description	Status	CVE/Issue id	CWE
XiangShan	B1: flh instruction does not perform sign extension.	Fixed	Issue #3979	CWE-681
	B2: Missing privilege access check for the stopi CSR in AIA.	Fixed	Issue #3978	CWE-284
	B3: Incorrect handling of custom CSRs in U-mode.	Fixed	Issue #3978	CWE-1220
	B4: Redundant checks for accessing S-mode custom CSRs in VS-mode.	Fixed	Issue #3978	CWE-1280
	B5: Interaction flaw between xstatus.SDT and sie.	Fixed	Issue #3934	CWE-1262
	B6: Incorrect rounding mode transition causes FMA result mismatch.	Fixed	CVE-2024-35028	CWE-682
	B7: High-bit truncation in SV39 bypasses PMP check.	Fixed	CVE-2024-35029	CWE-222
	B8: Special floating-point operand propagation causes register mismatch.	Fixed	CVE-2024-35027	CWE-1281
CVA6	B9: Access beyond the bounds of memory is permitted.	Confirmed	CVE-2024-35031	CWE-119
	B10: Rounding is calculated incorrectly under some microarchitectures.	Confirmed	CVE-2024-35032	CWE-682
	B11: Incorrectly setting the OF (Overflow Flag) bit for fsqrt calculations.	Confirmed	CVE-2024-35033	CWE-684
	B12: Incorrect smallest negative/negative infinity calculation.	Confirmed	CVE-2024-35034	CWE-682
	B13: The NX flag fails to be set under certain instruction sequences.	Confirmed	CVE-2024-35035	CWE-684
	B14: The zext.h instruction is implemented incorrectly.	Fixed	CVE-2024-35036	CWE-1281
NutShell	B15: The mtvec/stvec settings are incorrect.	Submitted	CVE-2024-22832	CWE-391
	B16: The mstatus register mishandles WPRI bits.	Submitted	CVE-2024-35017	CWE-440
	B17: Incorrect address truncation during memory access.	Submitted	Issue #215	CWE-222
	B18-19: Some atomic instructions incorrectly judge the sign bit.	Submitted	CVE-2024-35024	CWE-684
		Submitted	CVE-2024-35025	CWE-684
	B20: Registers are incorrectly updated during a page fault in M-mode.	Submitted	CVE-2024-35026	CWE-755
	B21: Incorrect Exception Handling in M-mode.	Submitted	CVE-2024-35016	CWE-754
Spike	B22: Incorrect implementation of the hvip LCOFI bit.	Fixed	Issue #1862	CWE-1262
Nemu	B23: Incorrect GVA setting in mstatus.	Fixed	Issue #642	CWE-1262
	B24: Incorrect dependency on values within the henvcfg.	Fixed	Issue #681	CWE-1254
	B25: hvip LCOFI bit incorrectly depends.	Fixed	Issue #687	CWE-1220
	B26: Incorrect access control dependency in mstateen0.	Fixed	Issue #690, #691	CWE-1262

Table 4: False positive elimination rate.

	XiangShan	Rocket	NutShell
Number of FP	11085	2308	12183
Remaining FP	1774	0	537
Elimination rate	84.0%	100%	95.6%

5.6 Security Analysis

We conducted a security analysis of selected bugs to evaluate their potential impact.

B5. Improper updates to the mstatus.SDT bit could lead to abnormal setting of sie, disrupting the proper management of interrupts and posing a serious security threat to the system. Specifically, if the mstatus.SDT bit is incorrectly updated to an invalid value under certain conditions, it may erroneously enable sie. This could inadvertently allow interrupts that should not be triggered, leading to privilege escalation in supervisor mode. Attackers could exploit this vulnerability by triggering unauthorized interrupts, bypassing the operating system’s security mechanisms, and gaining access to system resources or performing privileged operations that should otherwise be restricted. Particularly in virtualized environments, if an attacker gains supervisor-level access through this vulnerability, it could compromise the isolation between the host machine and

virtual machines, potentially leading to data leakage or tampering risks.

B4. Optional RISC-V extensions can create covert channels between different user threads or virtual machines. When CPU states introduced by these extensions (such as registers) are not visible to the host operating system or hypervisor, these states may not undergo context switches. As a result, a thread or operating system could modify the state, which could later be read by another thread, potentially leading to security vulnerabilities. This issue is especially critical in multi-core CPUs or virtualized environments, as they require isolation between different threads or virtual machines. If a malicious thread exploits these covert channels, it could bypass security mechanisms, enabling unauthorized access to information or resource misuse, and even leading to privilege escalation or data tampering. These bugs were found in the XiangShan CPU, and they have been fixed through our disclosure.

B14. Due to an incorrect implementation of the zext.h instruction, high-order bits may be incorrectly filled during instruction extension, leading to data corruption or information leakage. This issue is particularly critical in the context of cryptographic algorithms or sensitive data, as improper extension could alter the actual values of the data, potentially causing security checks to fail or sensitive information to be exposed. Attackers might exploit this flaw by providing specially crafted malicious inputs that cause the program to behave incorrectly, thus bypassing security measures and gaining unauthorized access or revealing sensitive data. This

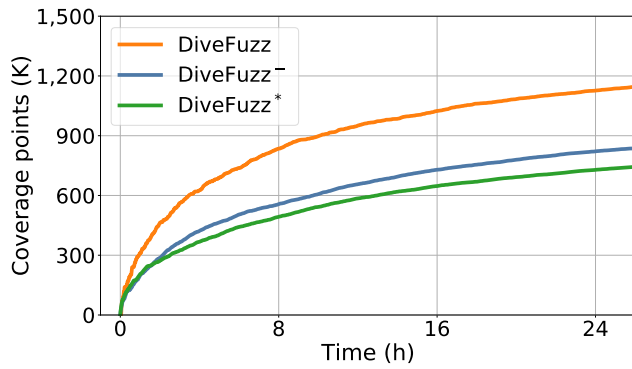


Figure 11: Comparison of control-coverage among DiveFuzz and its variants.

issue was identified in the CVA6 CPU, and has been fixed following our disclosure.

5.7 False Positive Elimination

We evaluated the fuzzer’s false positive elimination rate on XiangShan, Rocket, and NutShell, as shown in Table 4. During the evaluation, false positive issues and known bugs were handled uniformly. The reported false positives throughout the evaluation process are based on fuzzing results for 20,000 seeds. Although DiveFuzz’s false positive elimination mechanism did not resolve all seed issues on XiangShan and NutShell, it significantly reduced the workload for manual inspection. These false positives include access inconsistencies or atomic instruction issues caused by hardware characteristics, unflushed caches, repeated reports of known bugs, and differences in certain CSR implementations.

5.8 Component Analysis

To evaluate the key components of DiveFuzz, we constructed two variants of the tool. First, to assess the impact of instruction diversification on fuzzing effectiveness, we removed the diversification process and created DiveFuzz⁻. Then, we further removed both the diversification process and the feature in the mutator that balances instruction distribution, resulting in DiveFuzz^{*}. As shown in Figure 11, the coverage variation during fuzzing for these three versions of DiveFuzz is presented. The evaluation results indicate that instruction diversification significantly improved coverage during fuzzing, while balancing instruction distribution had a relatively limited effect on coverage improvement.

6 Discussion

The relationship between write-back data and the internal values of instruction operands. The root cause of bug detection in differential testing-based fuzzing methods lies in comparing the CPU’s write-back data with that of a golden model after executing instructions. The internal values of instruction operands directly determine the write-back data. To increase test coverage, DiveFuzz diversifies the write-back data by avoiding repetition in the

internal values of instruction operands. This method remains effective even for instructions with singular write-back data because it fundamentally avoids repeating the same instruction execution.

False positive and elimination. During fuzzing, false positives may occur due to differences in specific implementations. When mismatches arise during testing, manual intervention is required to determine whether they are false positives. Previous study [50] has suggested that false positives can be eliminated by synchronizing the inconsistent state between the REF and the CPU. However, this approach has limitations and can only be applied when the write-back data differs without causing a divergence in the program control flow. Many false positives lead to control flow divergences between the REF and the CPU, which cannot be resolved through state synchronization. Furthermore, avoiding the generation of such opcodes to prevent these false positives results in the loss of a significant number of test cases [45]. In contrast, DiveFuzz records false positives and known bugs as key-value pairs, and the fuzzer dynamically modifies the opcode of the instruction in real-time, allowing it to precisely avoid false positives and known bugs.

The role of coverage metrics in CPU fuzzing. In CPU fuzzing, coverage serves either as a real-time feedback signal to guide input generation—typical of coverage-guided fuzzers [27]—or as a post-hoc metric to evaluate test completeness, which is common in generation-based fuzzers [45]. While this paper primarily adopts the latter view, using coverage to assess final test effectiveness, DiveFuzz is not a purely generation-driven fuzzer. During input construction, it leverages feedback from the diversity of write-back data, focusing on instructions likely to increase such diversity. Experimental results show that this strategy improves traditional structural coverage and enables the discovery of previously unknown CPU bugs. Given the substantial differences in value space and architectural semantics across instruction types, future work may formalize abstract write-back data as a novel coverage metric to further enrich the CPU fuzzing ecosystem.

7 Related Work

Hardware fuzzing is a dynamic verification technique aimed at uncovering bugs and ensuring comprehensive coverage of hardware designs. This section reviews significant advancements and methodologies in the fuzzing domain, with a focus on tools and techniques pertinent to RISC-V CPUs. As shown in Table 5, a comparison of DiveFuzz with these fuzzers is provided.

Bitstream-based fuzzers. RFUZZ [31] was the first to introduce coverage-guided fuzzing to hardware, employing bit-level seed generation and mutation algorithms from AFL [20], such as bit-flipping. Its coverage metric is multiplexer coverage. Building on this, several works [11, 33] introduced directed fuzzing and symbolic execution to hardware fuzzing, scheduling seeds based on inter-module examples for targeted testing. However, these methods are less efficient at bug detection, as they rely on assertions for verification and have not uncovered new bugs. [49] also uses AFL for effective fuzzing. In contrast, Logic Fuzzer [28] randomizes input formats, focusing on specific states that do not affect functionality, thus increasing the microarchitectural state coverage, but adding complexity.

Table 5: Comparative analysis with previous hardware fuzzers.

Fuzzer	Input Generation Granularity	Mutation Strategy	Check Granularity	Coverage Comparison	New Bugs
RFUZZ [31]	Byte	AFL	Assertion	Baseline 1 (BL1)	0
DirectFuzz [11]	Byte	Instance distance	Assertion	2.23× faster than BL1	0
DifuzzRTL [27]	Instruction	Instruction segment	Program-level	Baseline 2 (BL2)	16
ProcessorFuzz [12]	Instruction	Instruction transformation	Program-level	1.23× faster than BL2	8
TheHuzz [29]	Instruction	Instruction field	Program-level	3.33× faster than BL2	11
Logic Fuzzer [28]	Byte	N/A	Instruction-level	N/A	13
MorFuzz [50]	Instruction	Instruction field, segment	Instruction-level	48× faster than BL2 (control register coverage)	17
SurgeFuzz [46]	Instruction	Predefined location	Program-level	Similar to BL2	5
Cascade [45]	Instruction control flow	N/A	Program-level	186× faster than BL2 (control register coverage)	37
DiveFuzz	Internal value of instruction operands	Instruction frequency, internal value of operands	Instruction-level	204× faster than BL2 114× faster than Cascade (five coverage metrics)	26

Assembly-based fuzzer. DifuzzRTL [27] was the first to detect real CPU bugs using fuzzing and introduced the definition of control register coverage metrics. Studies [50] have shown that this approach can be less effective than using randomly generated seeds for fuzzing. Building on DifuzzRTL, SurgeFuzz [46] implemented a directed fuzzing approach aimed at extensively testing edge cases and exceptional situations. ProcessorFuzz [12] introduced a new coverage metric based on CSR state transitions, successfully detecting new bugs. However, since they all use random instruction generation strategies, their final control register coverage metrics are similar to those of DifuzzRTL. TheHuzz [29] uses some commercial tools to collect coverage data, which has led to improvements in the final fuzzing results. However, due to its seed generation granularity and strategy, its code coverage is only 2.86% higher than that of random methods. MorFuzz [50] generates fuzzing seeds based on syntax and semantics and uses runtime information feedback to mutate instructions, resulting in final coverage that is 4.4 times higher than DifuzzRTL. Cascade [45] aims to enhance instruction execution efficiency by constructing long programs and eliminating control flow influences. It conducts the entire fuzzing process at the program-level granularity without using any mutation strategies to guide fuzzing. In addition, some open-source projects [4, 21, 41, 44] also provide independent testing platforms, which are designed to generate test cases for CPU testing. Since the instructions that trigger bugs have a direct dependency on the internal values of the operands, DiveFuzz achieves higher fuzz granularity by generating instructions at a fine-grained level to avoid executing instructions with the same internal operand values. This significantly increases the scenarios covered during the fuzzing process.

8 Conclusion

In this paper, we presented DiveFuzz, a novel CPU fuzzer designed to overcome the limitations of existing fuzzing techniques. By performing deeper checks on internal operand values and employing an advanced mutator for balanced opcode fuzzing and eliminating false positives, DiveFuzz significantly increases the diversity of instruction write-back data and improves fuzzing efficiency and

effectiveness. Our evaluations demonstrated that DiveFuzz outperforms state-of-the-art fuzzers, achieving coverage across five key metrics 204× faster than DifuzzRTL and 114× faster than Cascade. Additionally, DiveFuzz discovered 26 new bugs, 15 of which have been assigned CVE identifiers. These results highlight DiveFuzz’s capability in enhancing CPU security and reliability.

Acknowledgments

We thank the anonymous reviewers for their valuable feedback. We also appreciate the support from the XiangShan team, especially Prof. Yungang Bao (Institute of Computing Technology, Chinese Academy of Sciences) and Prof. Dan Tang (Beijing Institute of Open Source Chip). This research was supported by the Youth Innovation Promotion Association CAS, China and National Natural Science Foundation of China (No. 62172407).

References

- [1] 2024. The RISC-V Instruction Set Manual Volume I: Unprivileged Architecture. <https://github.com/riscv/riscv-isa-manual>.
- [2] 2024. The RISC-V Instruction Set Manual: Volume II: Privileged Architecture. <https://github.com/riscv/riscv-isa-manual>.
- [3] 2024. Verification Academy: Formal Verification. <https://verificationacademy.com/topics/formal-verification/>.
- [4] Chips Alliance. 2025. riscv-dv. <https://github.com/chipsalliance/riscv-dv>.
- [5] Krste Asanovic, Rimas Avizienis, Jonathan Bachrach, Scott Beamer, David Biancolin, Christopher Celio, Henry Cook, Daniel Dabbelt, John Hauser, Adam Izraelevitz, et al. 2016. The rocket chip generator. *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-17* 4 (2016), 6–2.
- [6] Krste Asanovic and David A Patterson. 2014. Instruction sets should be free: The case for risc-v. *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2014-146* (2014).
- [7] Nils Bars, Moritz Schloegel, Tobias Scharnowski, Nico Schiller, and Thorsten Holz. 2023. Fuzztruction: using fault injection-based fuzzing to leverage implicit domain knowledge. In *32nd USENIX Security Symposium (USENIX Security 23)*. 1847–1864.
- [8] Christopher M Bishop and Nasser M Nasrabadi. 2006. *Pattern recognition and machine learning*. Springer.
- [9] Niklas Bruns, Vladimir Herdt, Daniel Grosse, and Rolf Drechsler. 2022. Efficient cross-level processor verification using coverage-guided fuzzing. In *Proceedings of the Great Lakes Symposium on VLSI 2022*. 97–103.
- [10] Niklas Bruns, Vladimir Herdt, Eyck Jentsch, and Rolf Drechsler. 2022. Cross-level processor verification via endless randomized instruction stream generation with coverage-guided aging. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1123–1126.

- [11] Sadullah Canakci, Leila Delshadtehrani, Furkan Eris, Michael Bedford Taylor, Manuel Egele, and Ajay Joshi. 2021. Directfuzz: Automated test generation for rtl designs using directed graybox fuzzing. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 529–534.
- [12] Sadullah Canakci, Chathura Rajapaksha, Leila Delshadtehrani, Anoop Nataraja, Michael Bedford Taylor, Manuel Egele, and Ajay Joshi. 2023. Processorfuzz: Processor fuzzing with control and status registers guidance. In *2023 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*. IEEE, 1–12.
- [13] Luca Cassano, Stefano Di Mascio, Alessandro Palumbo, Alessandra Menicucci, Gianluca Furano, Giuseppe Bianchi, and Marco Ottavi. 2022. Is risc-v ready for space? a security perspective. In *2022 IEEE International Symposium on Defect and Fault Tolerance in VLSI and Nanotechnology Systems (DFT)*. IEEE, 1–6.
- [14] Chen Chen, Rahul Kande, Nathan Nguyen, Flemming Andersen, Aakash Tyagi, Ahmad-Reza Sadeghi, and Jayavijayan Rajendran. 2023. HyPFuzz: Formal-Assisted Processor Fuzzing. In *32nd USENIX Security Symposium (USENIX Security 23)*. 1361–1378.
- [15] National Vulnerability Database. 2023. CVE-2023-34885. <https://www.cve.org/CVERecord?id=CVE-2023-34885>.
- [16] National Vulnerability Database. 2023. CVE-2023-34908. <https://www.cve.org/CVERecord?id=CVE-2023-34908>.
- [17] Calvin Deutschbein, Andres Meza, Francesco Restuccia, Ryan Kastner, and Cynthia Sturton. 2021. Isadora: Automated information flow property generation for hardware designs. In *Proceedings of the 5th Workshop on Attacks and Solutions in Hardware Security*. 5–15.
- [18] Alexandru Dinu and Petre Lucian Ogrutan. 2022. Reinforcement Learning Made Affordable for Hardware Verification Engineers. *Micromachines* 13, 11 (2022), 1887.
- [19] RISC-V Foundation. 2024. RISC-V Specifications. <https://riscv.org/technical/specifications/>.
- [20] Google. 2020. American fuzzy lop. <https://github.com/google/AFL>.
- [21] OpenHW Group. 2022. force-riscv. <https://github.com/openhwgroup/force-riscv>.
- [22] Zihui Guo, Yin Lv, Ningning Cui, Liwei Chen, and Gang Shi. 2025. HScheduler: An execution history-based seed scheduling strategy for hardware fuzzing. *Computers & Security* (2025), 104479.
- [23] Osman Hasan and Sofiene Tahar. 2015. Formal verification methods. In *Encyclopedia of Information Science and Technology, Third Edition*. IGI global, 7162–7170.
- [24] Heqing Huang, Yiyuan Guo, Qingkai Shi, Peisen Yao, Rongxin Wu, and Charles Zhang. 2022. Beacon: Directed grey-box fuzzing with provable path pruning. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 36–50.
- [25] Heqing Huang, Anshunkang Zhou, Mathias Payer, and Charles Zhang. 2024. Everything is Good for Something: Counterexample-Guided Directed Fuzzing via Likely Invariant Inference. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 142–142.
- [26] Jaewon Hur, Suhwan Song, Sunwook Kim, and Byoungyoung Lee. 2022. SpecDoctor: Differential fuzz testing to find transient execution vulnerabilities. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*. 1473–1487.
- [27] Jaewon Hur, Suhwan Song, Dongup Kwon, Eunjin Baek, Jangwoo Kim, and Byoungyoung Lee. 2021. Difuzzrtl: Differential fuzz testing to find CPU bugs. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1286–1303.
- [28] Nursultan Kabytkas, Tommy Thorn, Shreesha Srinath, Polychronis Xekalakis, and Jose Renau. 2021. Effective processor verification with logic fuzzer enhanced co-simulation. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 667–678.
- [29] Rahul Kande, Addison Crump, Garrett Persyn, Patrick Jauernig, Ahmad-Reza Sadeghi, Aakash Tyagi, and Jayavijayan Rajendran. 2022. TheHuzz: Instruction fuzzing of processors using Golden-Reference models for finding Software-Exploitable vulnerabilities. In *31st USENIX Security Symposium (USENIX Security 22)*. 3219–3236.
- [30] Kevin Laeuffer, Vighnesh Iyer, David Biancolin, Jonathan Bachrach, Borivoje Nikolic, and Koushik Sen. 2023. Simulator independent coverage for RTL hardware languages. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 606–615.
- [31] Kevin Laeuffer, Jack Koenig, Donggyu Kim, Jonathan Bachrach, and Koushik Sen. 2018. RFUZZ: Coverage-directed fuzz testing of RTL on FPGAs. In *2018 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 1–8.
- [32] Gwangmu Lee, Woonchul Shim, and Byoungyoung Lee. 2021. Constraint-guided directed greybox fuzzing. In *30th USENIX Security Symposium (USENIX Security 21)*. 3559–3576.
- [33] Tun Li, Hongji Zou, Dan Luo, and Wanxia Qu. 2021. Symbolic simulation enhanced coverage-directed fuzz testing of rtl design. In *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 1–5.
- [34] Dian-Lun Lin, Yanqing Zhang, Haoxing Ren, Bruce Khailany, Shih-Hsin Wang, and Tsung-Wei Huang. 2023. Genfuzz: Gpu-accelerated hardware fuzzing using genetic algorithm with multiple inputs. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [35] Hangtian Liu, Shuitao Gan, Chao Zhang, Zicong Gao, Hongqi Zhang, Xiangzhi Wang, and Guangming Gao. 2024. LABRADOR: Response Guided Directed Fuzzing for Black-box IoT Devices. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 127–127.
- [36] LowRISC. 2025. Ibex: Cosim. <https://github.com/lowRISC/ibex/tree/master/dv/cosim>.
- [37] Tao Lu. 2021. A survey on risc-v security: Hardware and architecture. *arXiv preprint arXiv:2107.04175* (2021).
- [38] Chengfei Lv, Chaoyue Niu, Renjie Gu, Xiaotang Jiang, Zhaode Wang, Bin Liu, Ziqi Wu, Qiulin Yao, Congyu Huang, Panos Huang, et al. 2022. Walle: An End-to-End, General-Purpose, and Large-Scale Production System for Device-Cloud Collaborative Machine Learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. 249–265.
- [39] OpenXiangShan. 2025. DiffTest. <https://github.com/OpenXiangShan/difftest>.
- [40] Simon Resch, Juliane Friedrich, Timo Wagner, Gert Mehlmann, and Matthias Luther. 2021. Stability analysis of power hardware-in-the-loop simulations for grid applications. *Electronics* 11, 1 (2021), 7.
- [41] UC Berkeley Architecture Research. 2024. RISC-V torture test. <https://github.com/ucb-bar/riscv-torture>.
- [42] Wilson Snyder. 2025. Verilator. <https://github.com/verilator/verilator>.
- [43] RISC-V Software. 2025. riscv-isa-sim. <https://github.com/riscv-software-src/riscv-isa-sim>.
- [44] RISC-V Software. 2025. riscv-tests. <https://github.com/riscv-software-src/riscv-tests>.
- [45] Flavien Solt, Katharina Ceesay-Seitz, and Kaveh Razavi. 2024. Cascade: CPU Fuzzing via Intricate Program Generation. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA.
- [46] Yuichi Sugiyama, Reoma Matsuo, and Ryota Shioya. 2023. SurgeFuzz: Surge-Aware Directed Fuzzing for CPU Designs. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*. IEEE, 1–9.
- [47] OpenXiangShan Team. 2024. NEMU(NJU Emulator). <https://github.com/OpenXiangShan/NEMU>.
- [48] OSCP Team. 2024. Nutshell, risc-v cpu developed by oscpu team. <https://github.com/OSCPU/NutShell>.
- [49] Timothy Trippel, Kang G Shin, Alex Chernyakhovsky, Garret Kelly, Dominic Rizzo, and Matthew Hicks. 2022. Fuzzing hardware like software. In *31st USENIX Security Symposium (USENIX Security 22)*. 3237–3254.
- [50] Jinyan Xu, Yiyuan Liu, Sirui He, Haoran Lin, Yajin Zhou, and Cong Wang. 2023. MorFuzz: Fuzzing Processor via Runtime Instruction Morphing enhanced Synchronizable Co-simulation. In *32nd USENIX Security Symposium (USENIX Security 23)*. 1307–1324.
- [51] Yiru Xu, Hao Sun, Jianzhong Liu, Yuheng Shen, and Yu Jiang. 2024. SATURN: Host-Gadget Synergistic USB Driver Fuzzing. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 51–51.
- [52] Yinan Xu, Sa Wang, Dan Tang, Ninghui Sun, and Yungang Bao. 2024. PathFuzz: Broadening Fuzzing Horizons with Footprint Memory for CPUs. In *Proceedings of the 61st ACM/IEEE Design Automation Conference (DAC)*. 1–6.
- [53] Yinan Xu, Zihao Yu, Dan Tang, Guokai Chen, Lu Chen, Lingrui Gou, Yue Jin, Qianruo Li, Xin Li, ZuoJun Li, et al. 2022. Towards developing high performance RISC-V processors using agile methodology. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 1178–1199.
- [54] Florian Zaruba and Luca Benini. 2019. The cost of application-class processing: Energy and performance analysis of a Linux-ready 1.7-GHz 64-bit RISC-V core in 22-nm FDSOI technology. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 27, 11 (2019), 2629–2640.

A Template Introduction

Test templates. Initializing the environment is a critical step in fuzzing to ensure comprehensive and accurate test coverage. DiveFuzz’s test template includes the following operations: setting the CPU’s privilege levels (such as user mode, supervisor mode, machine mode, and hypervisor mode), configuring the page table for virtual memory mapping (DiveFuzz supports Sv39: a page-based 39-bit virtual memory system), configuring the physical memory protection (PMP) registers to control memory access permissions, initializing the trap vector base address register to define the entry point for exception handlers, randomly initializing general-purpose registers, and initializing data segments to ensure the diversity and randomness of data in memory.

Instruction templates. DiveFuzz supports a series of RISC-V ISA extensions, enabling testing across both 32-bit and 64-bit architectures. It covers base integer operations (RV32I/RV64I), multiplication and division (RV32M/RV64M), atomic memory operations (RV32A/RV64A), and floating-point arithmetic including single- and double-precision (RV32F/RV64F, RV32D/RV64D). It also includes support for compressed instructions (C extension) to enhance code density, bit-manipulation and cryptographic extensions for specialized computation, register access control to validate permission handling, and system-level instructions to ensure proper control and state management.

B CPU under Test

XiangShan [53] is currently the best-performing open-source RISC-V CPU, developed by the Chinese Academy of Sciences and written in Chisel. It stands out for its advanced feature support, including vector extensions and hypervisor capabilities. With its 13-stage out-of-order execution pipeline, XiangShan maximizes instruction-level parallelism, significantly improving throughput and overall system performance.

CVA6 [54], an open-source 64-bit in-order RISC-V CPU core affiliated with the OpenHW Group, is written in SystemVerilog. Despite its six-stage pipeline being single-issue, it has independent internal execution units capable of retiring multiple instructions simultaneously. CVA6 supports the base RISC-V instruction set and multiple extensions and has been fabricated using 22nm process technology.

Rocket [5], developed by UC Berkeley, is the world's first open-source RISC-V CPU, written in Chisel. It features a five-stage single-issue in-order scalar CPU architecture. This CPU supports the base RISC-V instruction set and multiple extensions, including virtualization and cryptographic extensions. It has been taped out multiple times and extensively validated by academia and industry.

NutShell [48] is an open-source 64-bit RISC-V CPU core developed under the "ysyx" project by the University of Chinese Academy of Sciences and the Institute of Computing Technology. Written in Chisel, it features a nine-stage pipeline design. NutShell supports the base RISC-V instruction set and multiple extensions, including I, M, A, C, Zicsr, and Zifencei. It has been successfully applied in various projects and has been fabricated.

C Bug Reproduction Across Fuzzers and Ablation Study

To evaluate the bug reproduction capability of different fuzzers, we configured DifuzzRTL and Cascade to run on the same target designs, and also included a variant of DiveFuzz with the write-back data diversification feature disabled (referred to as DiveFuzz⁻). Specifically, we used test seeds generated by DifuzzRTL, Cascade, and DiveFuzz, and executed them within DiveFuzz's framework. As shown in Table 6, both DifuzzRTL and Cascade successfully reproduced 3 out of the 26 known bugs. In the table, the fastest reproducing fuzzer for each bug is highlighted with an orange background.

Beyond its advantage in the number of discovered bugs, the effectiveness of the write-back data diversity mechanism can be further illustrated through concrete case studies. We analyze B11

Table 6: Comparison of bug discovery capabilities across fuzzers under the same design. DiveFuzz⁻ denotes a variant of DiveFuzz with the writeback-data diversity feature disabled. X indicates that the corresponding bug could not be reproduced within twenty-four hours of fuzzing. Cells highlighted in orange indicate the fastest bug reproduction time among all evaluated fuzzers.

Bug ID	Design	DifuzzRTL	Cascade	DiveFuzz ⁻	DiveFuzz
B1	XiangShan	X	X	21.3h	8.0h
B2	XiangShan	X	X	8.1h	6.4h
B3	XiangShan	X	X	X	4.3h
B4	XiangShan	X	X	X	14.7h
B5	XiangShan	X	X	X	14.7h
B6	XiangShan	X	5.0h	3.6h	2.4h
B7	XiangShan	X	X	13.8h	9.2h
B8	XiangShan	X	6.9h	8.5h	7.3h
B9	CVA6	X	X	1.8h	1.3h
B10	CVA6	X	X	X	9.3h
B11	CVA6	X	X	X	5.1h
B12	CVA6	X	X	X	54m
B13	CVA6	X	X	X	2.8h
B14	CVA6	X	X	5m	17m
B15	NutShell	X	X	X	1.3h
B16	NutShell	X	X	X	42m
B17	NutShell	X	X	1.2h	3.5h
B18	NutShell	23.7h	X	53m	13m
B19	NutShell	20.1h	X	34m	16m
B20	NutShell	X	1.2h	3.4h	1.3h
B21	NutShell	18.9h	X	4.1h	58m
B22	Spike	X	X	X	5.0h
B23	Nemu	X	X	X	7.2h
B24	Nemu	X	X	X	4.1h
B25	Nemu	X	X	X	5.0h
B26	Nemu	X	X	48m	53m

to demonstrate its impact. In the case of B11, which involves the `fsqrt` (floating-point square root) instruction, DiveFuzz enhances input diversity by recording the Shift-XOR patterns between source register values during instruction generation. In this test, DiveFuzz executed approximately 70,000 instances of the `fsqrt` instruction, among which over 40,000 used distinct, non-redundant operands. This resulted in a comparable number of unique write-back values, indicating that DiveFuzz was able to explore a wide variety of `fsqrt` execution scenarios. In contrast, although DifuzzRTL executed a similar order of magnitude of `fsqrt` instructions, it produced only a few hundred distinct write-back values concentrated in a limited set of fixed scenarios and thus failed to effectively trigger the B11 bug. For Cascade, its built-in bug circumvention mechanism prevented generation of `fsqrt` instructions in the default configuration; further analysis shows that even if Cascade were configured to generate such instructions, its write-back diversity would be comparable to that of DifuzzRTL.